PS1 <-> Python JSON-Lines Protocol and Implementation Plan

REVISED COMPLETE SPECIFICATION

Revision purpose

This revision fixes the coverage gap between the existing 47-page protocol/work-plan PDF and the final decisions recorded in **PrototypeWorflowAndDesign(1).txt**. The original protocol analysis is preserved unchanged; this revision adds the missing application-level contract and explicitly defines which statements control when earlier exploratory text conflicts with later approved decisions.

Authority and precedence

The source TXT begins with the instruction to use only the final structure of each step. Therefore, later approved decisions supersede earlier proposals. In this revised PDF, the following precedence applies:

- **1. Final approved decisions in PrototypeWorflowAndDesign(1).txt.**
- **2. The correction/addendum pages in this revised PDF.**
- 3. The original 47-page protocol/work-plan body, where it does not conflict with items 1 or 2.
- 4. Earlier exploratory alternatives in the TXT are historical context only and are not implementation requirements.

Critical corrected rule

> **When the user presses Post, the finalized local post record is saved/exported before Python is started. A remote X failure does not erase the local record. This is the final ordering and overrides any earlier proposal to export only after X confirms success.**

Scope of this revision

The added sections cover the missing GUI behavior, Post-button state logic, Attach/Remove behavior, Normal/Repost startup, manual Repost behavior, local history/export semantics, the exact configuration contract, the completed implementation work order, unresolved external/API questions, and a three-pass implementation review gate.

Important status labels

Label	Meaning
LOCKED	Explicitly approved/finalized in the source TXT and should not be redesigned during coding.
IMPLEMENTATION DECISION	Chosen here to make the approved design executable; may be changed only if it conflicts with a later user decision.
VERIFY BEFORE CODING	Depends on an external file/API/runtime behavior that the TXT does not fully define.

ChatGPT

PS1 ↔ Python JSON-Lines Protocol and Implementation Plan

Executive summary

The final architecture should use **one PowerShell process, one Python process, and three redirected standard streams**:

```
PowerShell → Python stdin   : one complete POST request
Python → PowerShell stdout  : JSON-lines protocol events
Python → PowerShell stderr  : diagnostics/debug only
```

There is **no need for one port per media file, no sockets, and no temporary request/response transport files**. The protocol is naturally suited to a single ordered stream because every media event carries its own `MediaId` and `FileIndex`. The application design already establishes that PowerShell owns the GUI/workflow, Python owns X communication, selected media are only local paths until Post is pressed, uploads are sequential, and the final app supports up to four media items. filecite turn0file0

The transport will use **JSON Lines**: one complete JSON object per physical line, UTF-8 encoded. JSON Lines explicitly uses one valid JSON value per line, while RFC 8259 defines the JSON representation itself. Python's own documentation also warns that ordinary JSON is not a framed protocol, which is exactly why adding a newline boundary around independently serialized objects is useful here. [1]

The locked event vocabulary is:

```
READY
TEXT_COMPLETE
PROGRESS
MEDIA_COMPLETE
POSTING
SUCCESS
ERROR
```

The most important distinction is:

```
REQUEST
= complete post
  Text
  TotalFiles
```

```
  Media[all selected files]

PROGRESS
= update for one media item
```

Thus, when `TotalFiles=2`, **both media entries are present in the initial request**. A later `PROGRESS` event mentions only the media file whose progress changed.

The expected successful sequence is:

```
PS1 starts Python
PS1 sends one complete request
    ↓
READY
    ↓
TEXT_COMPLETE
    ↓
PROGRESS MediaId=1
PROGRESS MediaId=1
...
MEDIA_COMPLETE MediaId=1
    ↓
PROGRESS MediaId=2
...
MEDIA_COMPLETE MediaId=2
    ↓
POSTING
    ↓
SUCCESS
```

`ERROR` is terminal and may replace the rest of that sequence at any stage.

This design matches the existing configuration and does not rename the values you have already locked:

```
X_Config.txt
Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt

PythonPath=C:\Python\Python313\python.exe
PythonScriptPath=Z:\InsPro\Tools\Py\XPoster.py
DataRootPath=Z:\InsPro\Tools\Data\X_Post
OperationalPath=Z:\InsPro\Tools\Data\X_Post\Operational
TextFilesPath=Z:\InsPro\Tools\Data\X_Post\TextFiles
MediaFilesPath=Z:\InsPro\Tools\Data\X_Post\MediaFiles
ExcelFilePath=Z:\InsPro\Tools\Data\X_Post\ExcelFile
KeysFilePath=Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt
```

```
DebugLogReadMePath=Z:\InsPro\Tools\Ps1\ReadMe\DebugLogReadMe.txt
DebugLogFunctionPath=Z:\InsPro\Tools\Ps1\DebugLog.ps1
DebugLogMode=Per run
DebugLogOutputPath=Z:\InsPro\Tools\Logs\XPoster\xxxxxxx.log
```

Those paths and the existing controller/function architecture come directly from the final design record. filecite turn0file0

Protocol and schema specification

Protocol identity

```
Name: XPoster PS1↔Python Protocol
ProtocolVersion: 1
Encoding: UTF-8
Record framing: one JSON object per line
PS1→PY channel: stdin
PY→PS1 protocol channel: stdout
PY→PS1 diagnostic channel: stderr
Media execution model: sequential
Maximum media count: 4
```

JSON Lines uses a newline as the record delimiter; records themselves must therefore remain compact JSON rather than pretty-printed multiline JSON. Newlines inside `Text` are not a problem because JSON serializers escape them inside the JSON string. [2]

For example, the human text:

```
first line
second line
```

appears on the wire inside one JSON record as conceptually:

```
{"Text":"first line\nsecond line"}
```

Python's `json.dumps()` produces JSON text, and `json.loads()` parses JSON text; compact separators can be used so each protocol message stays on one physical line. [3]

Full PS1 → Python request schema

The request is **one object containing the complete post**.

Field	Type	Required	Rule
`ProtocolVersion`	integer	Yes	Must equal `1`
`RequestId`	string	Yes	Non-empty ID generated by PS1; unique for the posting attempt
`Text`	string	Yes	May be empty
`TotalFiles`	integer	Yes	`0..4`
`Media`	array	Yes	Array length must exactly equal `TotalFiles`
`Media[].MediaId`	integer	Yes	Unique local protocol ID, normally `1..4`
`Media[].FileIndex`	integer	Yes	Sequential display/upload position, `1..TotalFiles`
`Media[].FileName`	string	Yes	Display filename, not `FileName1`, `FileName2`, etc.
`Media[].FilePath`	string	Yes	Absolute source path available to Python

The application-level rule remains that `Text` may be empty and `Media` may be empty, but not both. PowerShell normally prevents the invalid both-empty case by disabling Post; Python validates the same rule defensively. That behavior was already locked in the prototype design. filecite turn0file0

A complete two-file request is therefore:

```
{
  "ProtocolVersion": 1,
  "RequestId": "20261017_090153_001",
  "Text": "ddddddddddd",
  "TotalFiles": 2,
  "Media": [
    {
      "MediaId": 1,
      "FileIndex": 1,
      "FileName": "image1.jpg",
      "FilePath": "Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image1.jpg"
    },
    {
      "MediaId": 2,
      "FileIndex": 2,
      "FileName": "image2.jpg",
      "FilePath": "Z:\\InsPro\\Tools\\Data\\X_Post\\MediaFiles\\image2.jpg"
    }
```

```
  ]
}
```

That formatting is for documentation. **On the actual pipe it is one line:**

```
{"ProtocolVersion":
1,"RequestId":"20261017_090153_001","Text":"ddddddddddd","TotalFiles":2,"Media":
[{"MediaId":1,"FileIndex":1,"FileName":"image1.jpg","FilePath":"Z:\\InsPro\
\Tools\\Data\\X_Post\\MediaFiles\\image1.jpg"},{"MediaId":2,"FileIndex":
2,"FileName":"image2.jpg","FilePath":"Z:\\InsPro\\Tools\\Data\\X_Post\
\MediaFiles\\image2.jpg"}]}
```

There is deliberately no `FileName1`, `FileName2`, `FileName3`, or `FileName4`. `MediaId` and `FileIndex` already identify the item, so numbered property names would make parsing and future maintenance harder.

`RequestId`

The protocol treats the value as an opaque string. A readable PS1-generated form such as this is sufficient for the current single-process application:

```
20261017_090153_001
```

Every event relating to this operation echoes that same `RequestId`. Therefore stale output or output from the wrong Python process cannot silently update the wrong GUI operation.

`MediaId` **versus** `XMediaId`

These are deliberately different concepts:

```
MediaId
= local identity assigned before Python starts
= stays constant for the posting attempt

XMediaId
= media identity returned to Python by the upload layer
= not known until MEDIA_COMPLETE
```

The PS1 state should therefore maintain a mapping conceptually equivalent to:

```
MediaId 1 → XMediaId "..."
MediaId 2 → XMediaId "..."
```

```
MediaId 3 → XMediaId "..."
MediaId 4 → XMediaId "..."
```

Event schema

Event	Required fields	Optional fields	Meaning
`READY`	`Event`, `RequestId`, `ProtocolVersion`	none	Python parsed and accepted the complete request
`TEXT_COMPLETE`	`Event`, `RequestId`, `TextPresent`, `CharacterCount`	none	Text preparation/ validation finished
`PROGRESS`	`Event`, `RequestId`, `MediaId`, `FileIndex`, `TotalFiles`, `Percent`, `FileName`	`BytesUploaded`, `BytesTotal`	Realtime update for one media item
`MEDIA_COMPLETE`	`Event`, `RequestId`, `MediaId`, `FileIndex`, `TotalFiles`, `FileName`, `XMediaId`	none	One media upload finished
`POSTING`	`Event`, `RequestId`, `MediaCount`	none	Final post creation is beginning
`SUCCESS`	`Event`, `RequestId`, `PostId`, `PostUrl`	none	Final post was created
`ERROR`	`Event`, `RequestId`, `Code`, `Stage`, `Message`, `Retryable`	`MediaId`, `FileIndex`, `HttpStatus`	Terminal failure

`RequestId` may be `null` only in the unusual case where the input itself was so malformed that Python could not retrieve a request ID.

READY

```
{"Event":"READY","RequestId":"20261017_090153_001","ProtocolVersion":1}
```

`READY` does **not** mean merely “the Python executable started.” It means:

```
Python started
+
received the request
```

```
+
parsed JSON
+
accepted ProtocolVersion
+
validated the request structure
```

That removes an ambiguity in the earlier design.

TEXT_COMPLETE

```
{"Event":"TEXT_COMPLETE","RequestId":"20261017_090153_001","TextPresent":true,"CharacterCount":
11}
```

For a media-only post:

```
{"Event":"TEXT_COMPLETE","RequestId":"20261017_090153_001","TextPresent":false,"CharacterCount":
0}
```

This event should **always be emitted on a valid request**, including a media-only post. That gives PS1 a predictable state machine.

Its meaning is important:

> `TEXT_COMPLETE` means Python has accepted/prepared the text for the pending post. It does not mean the text has already been published to X.

Actual publication occurs at `POSTING`.

PROGRESS

```
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":37,"FileName":"image1.jpg"}
```

Later, still on the **same stdout stream**:

```
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":79,"FileName":"image1.jpg"}
```

Then file two:

```
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":2,"FileIndex":
2,"TotalFiles":2,"Percent":18,"FileName":"image2.jpg"}
```

There is no missing second media description here. The complete second media description was already sent in the request. `PROGRESS` intentionally describes only the media whose state changed.

Optional future-compatible byte fields may be emitted when the upload implementation has actual byte counts:

```
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":2,"FileIndex":
2,"TotalFiles":2,"Percent":18,"FileName":"image2.jpg","BytesUploaded":
1887436,"BytesTotal":10485760}
```

PS1 v1 does not require those optional values.

MEDIA_COMPLETE

```
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_001","MediaId":
1,"FileIndex":1,"TotalFiles":
2,"FileName":"image1.jpg","XMediaId":"123456789012345678"}
```

PS1 stores:

```
MediaIds[1] = "123456789012345678"
```

A later second completion:

```
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_001","MediaId":
2,"FileIndex":2,"TotalFiles":
2,"FileName":"image2.jpg","XMediaId":"987654321098765432"}
```

PS1 then has:

```
MediaIds[1] = "123456789012345678"
MediaIds[2] = "987654321098765432"
```

IDs returned by an external service should be handled as **strings**, not converted to floating-point numeric values, so there is no risk of losing decimal precision in intermediate representations.

POSTING

```
{"Event":"POSTING","RequestId":"20261017_090153_001","MediaCount":2}
```

At this point:

```
Text preparation complete
Media upload(s) complete
XMediaIds collected
Final post operation beginning
```

SUCCESS

```
{"Event":"SUCCESS","RequestId":"20261017_090153_001","PostId":"2034567890123456789","PostUrl":"ht
x.com/i/web/status/2034567890123456789"}
```

`PostId` is also represented as a string.

PowerShell maps those values to the already chosen state names:

```
PublishedPostId
PublishedPostUrl
```

ERROR

```
{"Event":"ERROR","RequestId":"20261017_090153_001","Code":"MEDIA_UPLOAD_FAILED","Stage":"MediaUpl
upload failed.","Retryable":true,"MediaId":2,"FileIndex":2}
```

An HTTP-related failure can additionally contain:

```
{"Event":"ERROR","RequestId":"20261017_090153_001","Code":"RATE_LIMITED","Stage":"MediaUpload","M
service rate limit reached.","Retryable":true,"MediaId":2,"FileIndex":
2,"HttpStatus":429}
```

Credentials, tokens, request authorization headers, and raw secrets must never appear in `Message`.

Runtime, error, logging, and safety rules

The PowerShell process should use `System.Diagnostics.Process` with redirected stdin/stdout/stderr and `UseShellExecute=false`. Microsoft documents redirected standard streams for precisely this parent-child-process communication pattern, and `BeginOutputReadLine()` provides asynchronous line-

oriented delivery through `OutputDataReceived`. Microsoft also specifically warns not to mix synchronous and asynchronous reading on the same redirected stream. [4]

This is a particularly good match for the protocol because an `OutputDataReceived` callback naturally corresponds to **one JSON event line**. [5]

Python's subprocess documentation likewise describes stdin, stdout, and stderr as distinct process streams and supports text-mode/encoding behavior; although Python is the child in this application rather than the launcher, the same standard-process-stream model applies. [6]

Stream rules are locked as follows:

```
stdin
    PS1 → Python
    protocol input only

stdout
    Python → PS1
    protocol JSON only

stderr
    Python → PS1
    human-readable diagnostics only
```

Python must therefore never do this:

```
print("Uploading...")
print("hello")
print("debug here")
```

to ordinary stdout.

Instead:

```
stdout:
{"Event":"PROGRESS",...}

stderr:
Uploading image1.jpg...
```

This isolation is essential. A stray debug print on stdout would otherwise be indistinguishable from a malformed protocol message.

Buffering and flushing

Each Python event must be flushed immediately:

```
sys.stdout.write(json_line + "\n")
sys.stdout.flush()
```

or:

```
print(json_line, flush=True)
```

Launching Python with `-u` is also appropriate as a defensive measure, but explicit flushing remains part of the event emitter contract.

stdin lifetime

Version 1 accepts exactly **one posting request per Python process**:

```
Start Python
→ PS1 writes request line
→ PS1 flushes
→ PS1 closes stdin
→ Python processes request
→ Python exits after SUCCESS or ERROR
```

This avoids needing command multiplexing, cancellation commands, request queues, or long-lived worker semantics in v1.

If a later application needs a persistent Python daemon that handles many posts, that should be a separately versioned design rather than silently altering v1.

Event ordering

Valid successful sequence:

```
READY
TEXT_COMPLETE
[PROGRESS ...]
[MEDIA_COMPLETE ...]
POSTING
SUCCESS
```

For two files:

```
READY
TEXT_COMPLETE

PROGRESS MediaId=1
...
MEDIA_COMPLETE MediaId=1

PROGRESS MediaId=2
...
MEDIA_COMPLETE MediaId=2

POSTING
SUCCESS
```

For text only:

```
READY
TEXT_COMPLETE
POSTING
SUCCESS
```

For media only:

```
READY
TEXT_COMPLETE
PROGRESS
...
MEDIA_COMPLETE
POSTING
SUCCESS
```

`SUCCESS` and `ERROR` are **terminal events**. Python must produce no later protocol event after either one.

A well-behaved Python process should exit with:

```
0     after SUCCESS
non-0 after ERROR
```

The protocol event is still the primary semantic result; the process exit code provides an independent process-health check.

Error codes

The codes should remain readable rather than becoming opaque numbers.

Code	Typical stage	Retryable	Recovery
`MALFORMED_REQUEST_JSON`	RequestValidation	No	Fix protocol producer
`UNSUPPORTED_PROTOCOL_VERSION`	RequestValidation	No	Update PS1/Python pair
`INVALID_REQUEST`	RequestValidation	No	Fix request fields
`EMPTY_POST`	RequestValidation	No	Restore GUI; require text or media
`MEDIA_COUNT_INVALID`	RequestValidation	No	Rebuild request
`MEDIA_FILE_NOT_FOUND`	RequestValidation	No	User reselects/ restores file
`MEDIA_FILE_UNREADABLE`	RequestValidation	No	Fix access/path
`CONFIG_ERROR`	Startup	No	Correct `X_Config.txt`
`CREDENTIAL_ERROR`	Startup/Auth	No	Correct credentials
`AUTH_FAILED`	Auth	Usually no	Fix authentication
`RATE_LIMITED`	MediaUpload/ PostCreate	Yes	Retry later, manually in v1
`NETWORK_ERROR`	MediaUpload/ PostCreate	Yes	Manual retry in v1
`MEDIA_UPLOAD_FAILED`	MediaUpload	Depends	Abort post; manual retry
`POST_CREATE_FAILED`	PostCreate	Depends	Do not auto-repeat
`POST_RESULT_UNKNOWN`	PostCreate	No automatic retry	Reconcile before trying again
`INTERNAL_ERROR`	Internal	No automatic retry	Inspect diagnostics/ log

PowerShell also needs its own locally generated failures for situations where Python cannot emit an `ERROR` event:

```
PYTHON_START_FAILED
PYTHON_EXITED_UNEXPECTEDLY
PROTOCOL_INVALID_STDOUT_JSON
PROTOCOL_REQUEST_ID_MISMATCH
PROTOCOL_UNEXPECTED_EVENT
PROTOCOL_SEQUENCE_ERROR
OPERATION_TIMEOUT
```

Why `POST_RESULT_UNKNOWN` matters

There is a dangerous difference between:

```
"We know the post creation request failed."
```

and:

```
"The connection failed after sending the creation request, so we do not know
whether the remote side created the post."
```

The second condition must **not** trigger an automatic resend, because doing so can create a duplicate post. Therefore v1 has a deliberately conservative recovery rule:

> No automatic repost retries.

`Retryable=true` means that retry may make sense after inspection; it does not instruct PS1 to automatically repeat the operation.

Partial media upload

Suppose:

```
MediaId=1 → MEDIA_COMPLETE → XMediaId A
MediaId=2 → ERROR
```

The rule is:

```
Do not POST.
Discard the current posting operation.
Do not consider the partial XMediaId map a completed post.
Restore GUI for manual recovery.
```

A new Post action receives a **new RequestId** and, in v1, performs a fresh upload sequence.

That deliberately keeps recovery simple. Optimization through reuse of remote media IDs can be considered later, but should not complicate the initial implementation.

Malformed stdout

Because stdout is protocol-only:

```
This is debug output
```

is not harmless.

PS1 attempts `ConvertFrom-Json`; parsing fails; the operation becomes:

```
PROTOCOL_INVALID_STDOUT_JSON
```

The Python operation should then be terminated or considered failed. Continuing after losing protocol framing risks applying subsequent messages to an unknown state.

PowerShell's `ConvertFrom-Json` converts JSON text to PowerShell objects, while `ConvertTo-Json` performs the reverse conversion. [7]

stderr

stderr is not parsed as protocol.

Example:

```
[PY-STDERR] Upload retry attempt failed: timeout
```

PS1 sends that text to:

```
Debug / Output window
+
DebugLog
```

but it does not call `HandlePythonResponse` for it.

Reading stdout and stderr without blocking each other is important when both are redirected; Microsoft's Process documentation notes that asynchronous stream handling avoids the deadlock risks associated with blocking reads of redirected process streams. [8]

GUI threading

The asynchronous process-output callback must not directly manipulate Windows Forms controls from the process callback thread. Windows Forms controls are not generally thread-safe and Microsoft recommends marshaling updates to the owning UI thread with mechanisms such as `Invoke` / `BeginInvoke`. [9]

So conceptually:

```
OutputDataReceived thread
    ↓
HandlePythonResponse
    ↓
queue/marshal GUI action
    ↓
GUI thread
    ↓
UpdateUploadProgress
```

not:

```
OutputDataReceived thread
    ↓
directly modify progress bar
```

Logging rules

Use the logger already specified by config:

```
DebugLogFunctionPath=Z:\InsPro\Tools\Ps1\DebugLog.ps1
DebugLogMode=Per run
DebugLogOutputPath=Z:\InsPro\Tools\Logs\XPoster\xxxxxxx.log
```

Do not normalize the locked value `"Per run"` into some different config value.

Each useful protocol log record should include:

```
Timestamp
RequestId
Direction
Event or Stage
MediaId when applicable
FileIndex when applicable
```

```
Percent when applicable
ErrorCode when applicable
```

Example:

```
2026-10-17 09:10:01.152 [RequestId=20261017_090958_001] PY→PS PROGRESS MediaId=2
FileIndex=2 Percent=37
```

A `MEDIA_COMPLETE` line:

```
2026-10-17 09:10:04.886 [RequestId=20261017_090958_001] PY→PS MEDIA_COMPLETE
MediaId=2
```

Secrets are excluded.

Do **not** log:

```
API keys
API secrets
access tokens
refresh tokens
Authorization headers
complete X_Keys.txt contents
```

The request itself contains post text, so raw request logging should also be avoided by default. A safer summary is:

```
RequestId=...
TextLength=...
TotalFiles=...
MediaId=1 FileName=...
MediaId=2 FileName=...
```

Full local file paths may also be omitted from ordinary log entries unless needed for a specific diagnostic mode.

Credentials remain at:

```
Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt
```

and are **not part of the PS1→Python protocol**. This preserves the separation already established in the application design. filecite turn0file0

Process-launch safety

PowerShell should launch the configured executable directly, not through `cmd.exe`. `UseShellExecute` must be false for redirected streams. Python's process documentation likewise recommends explicit executable/argument handling and notes the security implications of invoking shell parsing unnecessarily. [10]

Thus:

```
GOOD
C:\Python\Python313\python.exe
    -u
    Z:\InsPro\Tools\Py\XPoster.py
```

rather than building an arbitrary command string passed through a shell.

Implementation plan and pseudocode

The implementation should be done in layers so that communication can be tested independently from the X API.

Work order

Phase	Implementation	Acceptance condition
Protocol constants	Event names, schemas, validation rules	PS1 and Python use identical names
Python protocol harness	Read request and emit fake events	PS1 receives realtime events without X API
PS1 transport	Start process, stdin request, async stdout/stderr	GUI remains responsive
PS1 event router	Parse/validate JSON and dispatch	All seven events route correctly
Progress state	`MediaId` map and GUI progress fields	File switching works 1→2→3→4
Python upload engine	Replace fake upload with real upload	`PROGRESS` / `MEDIA_COMPLETE` are real
Final posting	Emit `POSTING`, `SUCCESS`, `ERROR`	Post ID/URL reaches PowerShell
Logging	Correlated protocol/process log	One RequestId traceable end-to-end

Phase	Implementation	Acceptance condition
Fault testing	Crash, malformed data, partial upload	Deterministic recovery
Final integration	`StartPost` + GUI + export workflow	Full approved workflow passes

The existing PowerShell control boundary remains:

```
MainController
│
├─ ReadConfig
│
├─ GuiController
│  ├─ DetermineStartupMode
│  ├─ InitializeGui
│  ├─ AttachMedia
│  ├─ RemoveMedia
│  ├─ UpdatePostButtonState
│  ├─ UpdateGui
│  └─ UpdateUploadProgress
│
└─ StartPost
   ├─ ExportPostRecord
   ├─ RunPythonEngine
   └─ HandlePythonResponse
```

That is the final structure recorded in the uploaded design. filecite turn0file0

Building the request in PowerShell

Conceptual pseudocode:

```powershell
function New-PythonPostRequest {
    param(
        [string]$PostText,
        [array]$SelectedMedia
    )

    $requestId = Get-Date -Format 'yyyyMMdd_HHmmss_fff'

    $media = @()

    for ($i = 0; $i -lt $SelectedMedia.Count; $i++) {
        $path = [string]$SelectedMedia[$i]

        $media += [pscustomobject]@{
```

```
            MediaId   = $i + 1
            FileIndex = $i + 1
            FileName  = [System.IO.Path]::GetFileName($path)
            FilePath  = $path
        }
    }

    return [pscustomobject]@{
        ProtocolVersion = 1
        RequestId       = $requestId
        Text            = $PostText
        TotalFiles      = $media.Count
        Media           = $media
    }
}
```

Actual production code should validate before this object is sent:

```
ProtocolVersion = 1
Text OR Media present
TotalFiles <= 4
Media.Count == TotalFiles
MediaId unique
FileIndex sequential
all FilePath values exist
```

Launching Python and sending one JSON request

PowerShell pseudocode:

```
function RunPythonEngine {
    param(
        $Config,
        $Request,
        $AppState
    )

    $startInfo = [System.Diagnostics.ProcessStartInfo]::new()

    $startInfo.FileName = $Config.PythonPath

    # Conceptual argument:
    # python.exe -u "Z:\InsPro\Tools\Py\XPoster.py"
    $startInfo.Arguments = '-u "' + $Config.PythonScriptPath + '"'
```

```
$startInfo.UseShellExecute        = $false
$startInfo.CreateNoWindow         = $true
$startInfo.RedirectStandardInput  = $true
$startInfo.RedirectStandardOutput = $true
$startInfo.RedirectStandardError  = $true

$utf8 = [System.Text.UTF8Encoding]::new($false)

# Where supported by the target .NET runtime:
$startInfo.StandardInputEncoding  = $utf8
$startInfo.StandardOutputEncoding = $utf8
$startInfo.StandardErrorEncoding  = $utf8

$process = [System.Diagnostics.Process]::new()
$process.StartInfo = $startInfo
$process.EnableRaisingEvents = $true

$process.add_OutputDataReceived({
    param($sender, $eventArgs)

    if ($null -ne $eventArgs.Data) {
        HandlePythonResponse `
            -PythonMessage $eventArgs.Data `
            -ExpectedRequestId $AppState.RequestId `
            -AppState $AppState
    }
})

$process.add_ErrorDataReceived({
    param($sender, $eventArgs)

    if ($null -ne $eventArgs.Data) {
        WriteDebugOutput "[PY-STDERR] $($eventArgs.Data)"
    }
})

if (-not $process.Start()) {
    throw "PYTHON_START_FAILED"
}

$process.BeginOutputReadLine()
$process.BeginErrorReadLine()

$json = $Request | ConvertTo-Json -Depth 6 -Compress

$process.StandardInput.WriteLine($json)
$process.StandardInput.Flush()
$process.StandardInput.Close()
```

```
    return $process
}
```

Microsoft documents that redirection requires the corresponding `RedirectStandard...` properties and that asynchronous output reading is started with `BeginOutputReadLine()`. [11]

Crucially, the GUI thread should **not** then call a blocking `WaitForExit()` and freeze the form. Completion should be detected asynchronously through a process-exit event or equivalent worker mechanism.

Reading and validating stdout

```
function HandlePythonResponse {
    param(
        [string]$PythonMessage,
        [string]$ExpectedRequestId,
        $AppState
    )

    try {
        $message = $PythonMessage | ConvertFrom-Json -ErrorAction Stop
    }
    catch {
        StopPostingOperation `
            -Code "PROTOCOL_INVALID_STDOUT_JSON" `
            -Message $PythonMessage

        return
    }

    if ([string]::IsNullOrWhiteSpace($message.Event)) {
        StopPostingOperation `
            -Code "PROTOCOL_UNEXPECTED_EVENT" `
            -Message "Event field is missing."

        return
    }

    if ($message.RequestId -ne $ExpectedRequestId) {
        StopPostingOperation `
            -Code "PROTOCOL_REQUEST_ID_MISMATCH" `
            -Message "Unexpected RequestId."

        return
    }
```

```
    switch ($message.Event) {
        "READY" {
            $AppState.PythonResult = "Ready"
        }

        "TEXT_COMPLETE" {
            UpdateGui -GuiState "TextComplete"
        }

        "PROGRESS" {
            HandleProgressEvent `
                -Message $message `
                -AppState $AppState
        }

        "MEDIA_COMPLETE" {
            HandleMediaCompleteEvent `
                -Message $message `
                -AppState $AppState
        }

        "POSTING" {
            UpdateGui -GuiState "Posting"
        }

        "SUCCESS" {
            $AppState.PublishedPostId  = [string]$message.PostId
            $AppState.PublishedPostUrl = [string]$message.PostUrl
            $AppState.PythonResult     = "Success"
            $AppState.PostInProgress   = $false

            CompletePostWorkflow -AppState $AppState
        }

        "ERROR" {
            $AppState.PythonResult   = "Error"
            $AppState.PostInProgress = $false

            HandlePostingError -Message $message
        }

        default {
            StopPostingOperation `
                -Code "PROTOCOL_UNEXPECTED_EVENT" `
                -Message "Unknown event: $($message.Event)"
        }
    }
}
```

PowerShell's JSON conversion cmdlets are appropriate for converting protocol lines to/from PowerShell objects. [7]

PROGRESS state update

```
function HandleProgressEvent {
    param(
        $Message,
        $AppState
    )

    $AppState.CurrentFileIndex    = [int]$Message.FileIndex
    $AppState.TotalFiles          = [int]$Message.TotalFiles
    $AppState.CurrentFileProgress = [int]$Message.Percent
    $AppState.CurrentFileName     = [string]$Message.FileName

    # This operation must be marshaled to the GUI thread.
    InvokeGuiUpdate {
        UpdateUploadProgress `
            -CurrentFileIndex $AppState.CurrentFileIndex `
            -TotalFiles $AppState.TotalFiles `
            -CurrentFileProgress $AppState.CurrentFileProgress `
            -CurrentFileName $AppState.CurrentFileName
    }
}
```

That directly preserves your already-selected GUI state names:

```
CurrentFileIndex
TotalFiles
CurrentFileProgress
CurrentFileName
```

from the prototype design. 𝌆filecite𝌆turn0file0𝌆

MEDIA_COMPLETE mapping

```
function HandleMediaCompleteEvent {
    param(
        $Message,
        $AppState
    )

    $mediaId  = [int]$Message.MediaId
```

```
    $xMediaId = [string]$Message.XMediaId

    $AppState.MediaIds[$mediaId] = $xMediaId

    # Defensive UI normalization:
    $AppState.CurrentFileProgress = 100

    InvokeGuiUpdate {
        UpdateUploadProgress `
            -CurrentFileIndex ([int]$Message.FileIndex) `
            -TotalFiles ([int]$Message.TotalFiles) `
            -CurrentFileProgress 100 `
            -CurrentFileName ([string]$Message.FileName)
    }
}
```

Python request reader

Python pseudocode:

```
import json
import sys
from typing import Any


PROTOCOL_VERSION = 1


def read_request() -> dict[str, Any]:
    line = sys.stdin.readline()

    if line == "":
        raise ValueError("No request received on stdin.")

    try:
        request = json.loads(line)
    except json.JSONDecodeError as exc:
        raise ValueError(f"Malformed request JSON: {exc}") from exc

    validate_request(request)
    return request
```

Python's standard `json.loads()` decodes JSON strings; `json.dumps()` performs the inverse serialization. [3]

Python protocol emitter

```
import json
import sys
from typing import Any


def emit_event(
    event: str,
    request_id: str | None,
    **fields: Any,
) -> None:
    payload: dict[str, Any] = {
        "Event": event,
        "RequestId": request_id,
    }

    payload.update(fields)

    line = json.dumps(
        payload,
        ensure_ascii=False,
        separators=(",", ":"),
        allow_nan=False,
    )

    sys.stdout.write(line + "\n")
    sys.stdout.flush()
```

Using compact JSON rather than indentation is important because each protocol object must stay on one line. Python's `json` documentation explicitly distinguishes compact output from pretty printing and notes that JSON itself does not supply framing for multiple serialized objects. [3]

Python stderr helper

```
def debug(message: str) -> None:
    sys.stderr.write(message + "\n")
    sys.stderr.flush()
```

Never:

```
print("debug")
```

unless `file=sys.stderr` is explicitly supplied.

Python main sequence

```
def main() -> int:
    request_id: str | None = None

    try:
        request = read_request()
        request_id = request["RequestId"]

        emit_event(
            "READY",
            request_id,
            ProtocolVersion=PROTOCOL_VERSION,
        )

        text = request["Text"]

        prepare_text(text)

        emit_event(
            "TEXT_COMPLETE",
            request_id,
            TextPresent=bool(text),
            CharacterCount=len(text),
        )

        x_media_ids: list[str] = []

        for media in request["Media"]:
            media_id = media["MediaId"]
            file_index = media["FileIndex"]
            file_name = media["FileName"]
            file_path = media["FilePath"]

            def on_progress(percent: int) -> None:
                emit_event(
                    "PROGRESS",
                    request_id,
                    MediaId=media_id,
                    FileIndex=file_index,
                    TotalFiles=request["TotalFiles"],
                    Percent=percent,
                    FileName=file_name,
                )

            x_media_id = upload_media(
                file_path=file_path,
                progress_callback=on_progress,
            )
```

```
            x_media_ids.append(x_media_id)

            emit_event(
                "MEDIA_COMPLETE",
                request_id,
                MediaId=media_id,
                FileIndex=file_index,
                TotalFiles=request["TotalFiles"],
                FileName=file_name,
                XMediaId=x_media_id,
            )

        emit_event(
            "POSTING",
            request_id,
            MediaCount=request["TotalFiles"],
        )

        result = create_post(
            text=text,
            x_media_ids=x_media_ids,
        )

        emit_event(
            "SUCCESS",
            request_id,
            PostId=result.post_id,
            PostUrl=result.post_url,
        )

        return 0

    except KnownProtocolError as exc:
        emit_event(
            "ERROR",
            request_id,
            Code=exc.code,
            Stage=exc.stage,
            Message=exc.safe_message,
            Retryable=exc.retryable,
        )
        debug(exc.diagnostic_message)
        return 1

    except Exception as exc:
        emit_event(
            "ERROR",
```

```
            request_id,
            Code="INTERNAL_ERROR",
            Stage="Internal",
            Message="Unexpected Python engine error.",
            Retryable=False,
        )

        # Detailed traceback belongs on stderr/logging, never stdout.
        debug(repr(exc))
        return 1
```

Alternatives, end-to-end exchange, and edge cases

The transport decision is straightforward once the actual requirements are compared.

Design	Advantages	Disadvantages	Verdict
JSON Lines over stdin/ stdout	Structured fields; native parsers in PS/Python; proper escaping; one stream; easy `MediaId` routing; human-readable; easy versioning	stdout must remain protocol-only; producer must flush events	**Use this**
`EVENT|Key=Value`	Superficially simple; easy to type manually	Must invent escaping for `|`, `=`, newlines and Unicode; arrays become awkward; schema/ versioning weaker	Reject
Multiple sockets / port per file	Can separate concurrent producers	Port allocation, lifecycle, firewall concerns, synchronization, ordering, error handling and cleanup become much more complicated	Reject
One socket carrying all files	Could support long-running remote architecture	Still introduces networking when parent/child pipes already exist	Unnecessary for v1

The standard process APIs already provide stdin/stdout/stderr pipes, and Microsoft's `Process` APIs directly support asynchronous line reading from redirected output, so introducing network sockets would add machinery without solving a present requirement. [12]

Exact successful exchange

PS1 sends exactly one line:

```
{"ProtocolVersion":
1,"RequestId":"20261017_090153_001","Text":"ddddddddddd","TotalFiles":2,"Media":
[{"MediaId":1,"FileIndex":1,"FileName":"image1.jpg","FilePath":"Z:\\InsPro\
\Tools\\Data\\X_Post\\MediaFiles\\image1.jpg"},{"MediaId":2,"FileIndex":
2,"FileName":"image2.jpg","FilePath":"Z:\\InsPro\\Tools\\Data\\X_Post\
\MediaFiles\\image2.jpg"}]}
```


Python replies, one object per line:


```
{"Event":"READY","RequestId":"20261017_090153_001","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_090153_001","TextPresent":true,"CharacterCount":
11}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":0,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":37,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":81,"FileName":"image1.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":1,"FileIndex":
1,"TotalFiles":2,"Percent":100,"FileName":"image1.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_001","MediaId":
1,"FileIndex":1,"TotalFiles":
2,"FileName":"image1.jpg","XMediaId":"11111111111111111"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":2,"FileIndex":
2,"TotalFiles":2,"Percent":0,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":2,"FileIndex":
2,"TotalFiles":2,"Percent":42,"FileName":"image2.jpg"}
{"Event":"PROGRESS","RequestId":"20261017_090153_001","MediaId":2,"FileIndex":
2,"TotalFiles":2,"Percent":100,"FileName":"image2.jpg"}
{"Event":"MEDIA_COMPLETE","RequestId":"20261017_090153_001","MediaId":
2,"FileIndex":2,"TotalFiles":
2,"FileName":"image2.jpg","XMediaId":"22222222222222222"}
{"Event":"POSTING","RequestId":"20261017_090153_001","MediaCount":2}
{"Event":"SUCCESS","RequestId":"20261017_090153_001","PostId":"333333333333333333","PostUrl":"htt
x.com/i/web/status/333333333333333333"}
```


All of that comes through **one stdout stream**.

There are not two stdout pipes:

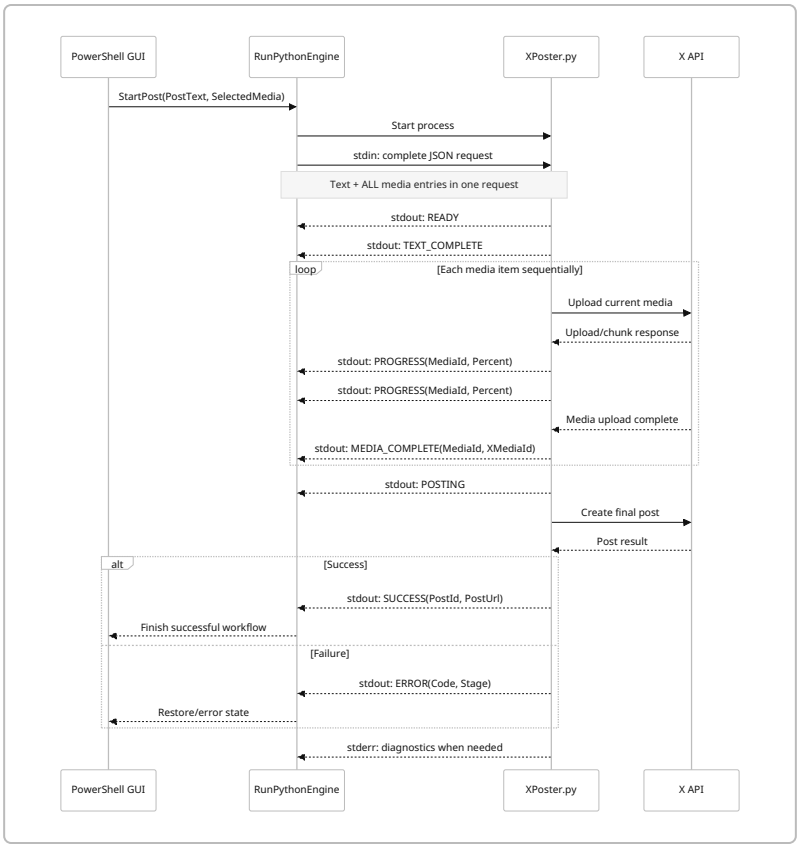
```
Media1 pipe
Media2 pipe
```

There is:

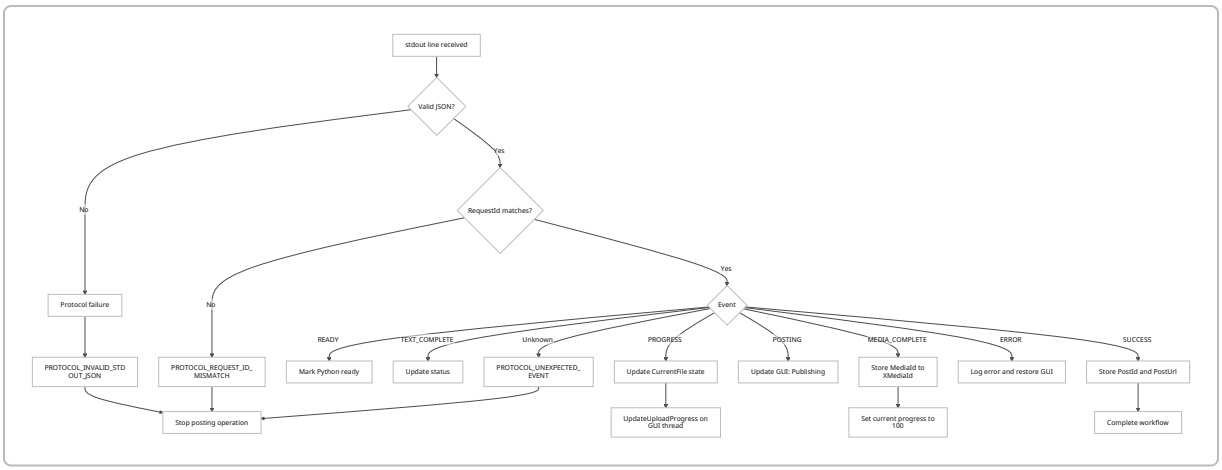
```
stdout
│
├─ READY
├─ TEXT_COMPLETE
├─ PROGRESS MediaId=1
├─ PROGRESS MediaId=1
├─ MEDIA_COMPLETE MediaId=1
├─ PROGRESS MediaId=2
├─ MEDIA_COMPLETE MediaId=2
├─ POSTING
└─ SUCCESS
```

The identifiers make the stream self-describing.

Sequence diagram



PS1 event-routing flow



Process crash

Example:

```
READY
TEXT_COMPLETE
PROGRESS MediaId=1 Percent=37

<python process terminates>
```

There is no `SUCCESS` and no `ERROR`.

PS1 synthesizes:

```
Code=PYTHON_EXITED_UNEXPECTEDLY
```

and records the process exit code plus recent stderr.

It must not assume the post failed in a way that is safe to retry automatically. In particular, if the crash occurred around final post creation, the remote outcome may require manual confirmation.

Malformed Python JSON

Python accidentally prints:

```
Uploading...
```

PS1 receives it on stdout.

Result:

```
PROTOCOL_INVALID_STDOUT_JSON
```

The stream is considered compromised and the operation is stopped.

Partial upload

Sequence:

```
MEDIA_COMPLETE MediaId=1 XMediaId=A
PROGRESS MediaId=2 Percent=61
ERROR MEDIA_UPLOAD_FAILED MediaId=2
```

Result:

```
No POSTING
No final post creation
No SUCCESS
```

PS1 keeps enough information for diagnostics but does not treat the first remote media ID as a completed post.

Parallel upload

Parallel upload is **not part of protocol v1**.

Version 1 explicitly means:

```
MediaId 1 finishes before MediaId 2 begins
MediaId 2 finishes before MediaId 3 begins
...
```

That matches the GUI design where the progress area “switches to the currently uploading file” and the existing assumption of sequential processing. filecite turn0file0

A future parallel implementation would technically still be able to use one stdout stream:

```
PROGRESS MediaId=1 Percent=40
PROGRESS MediaId=2 Percent=12
```

```
PROGRESS MediaId=1 Percent=51
PROGRESS MediaId=2 Percent=19
```

so parallelism still would **not require one socket per media file**. However, the GUI/state semantics would change enough that this should be treated as a future protocol/application capability rather than introduced accidentally.

Text-only post

Request:

```
{"ProtocolVersion":1,"RequestId":"20261017_091500_001","Text":"Text
only","TotalFiles":0,"Media":[]}
```

Events:

```
{"Event":"READY","RequestId":"20261017_091500_001","ProtocolVersion":1}
{"Event":"TEXT_COMPLETE","RequestId":"20261017_091500_001","TextPresent":true,"CharacterCount":
9}
{"Event":"POSTING","RequestId":"20261017_091500_001","MediaCount":0}
{"Event":"SUCCESS","RequestId":"20261017_091500_001","PostId":"...","PostUrl":"..."}
```

Media-only post

Request:

```
{"ProtocolVersion":1,"RequestId":"20261017_091600_001","Text":"","TotalFiles":
1,"Media":[{"MediaId":1,"FileIndex":1,"FileName":"photo.jpg","FilePath":"Z:\\...
\\photo.jpg"}]}
```

`TEXT_COMPLETE` is still present:

```
{"Event":"TEXT_COMPLETE","RequestId":"20261017_091600_001","TextPresent":false,"CharacterCount":
0}
```

Empty post

```
{"ProtocolVersion":1,"RequestId":"20261017_091700_001","Text":"","TotalFiles":
0,"Media":[]}
```

Normally PS1 never sends this because Post is disabled. If it somehow reaches Python, Python emits:

{"Event":"ERROR","RequestId":"20261017_091700_001","Code":"EMPTY_POST","Stage":"RequestValidation
must contain text or at least one media item.","Retryable":false}

That implements defense in depth without changing the GUI rule already established in the design.
filecite turn0file0

Review 1

Simulated user upload: the first complete protocol/work-plan draft is uploaded back for review.

I would review it as though I had never written it and look specifically for ambiguity between what is sent once and what is streamed later.

The first review finds four issues.

Finding: the original meaning of `READY` was too loose.

Earlier wording such as:

```
Python started correctly
```

is insufficient.

A Python executable can start and then fail to parse the request. PS1 should not show Ready in that case.

Revision:

`READY` is now explicitly:

```
Python started
AND
request received
AND
request JSON parsed
AND
ProtocolVersion accepted
AND
request schema accepted
```

This eliminates a race/semantic ambiguity.

Finding: `TEXT_COMPLETE` could be misunderstood as "text has been posted."

The actual final post is not created until all media IDs are ready.

Revision:

```
TEXT_COMPLETE
= local Python preparation/validation of Text is complete
≠ remote X post has been created
```

It is always emitted, including when `Text=""`, so the event sequence remains deterministic.

Finding: the earlier `PROGRESS` example created exactly the confusion the user identified.

This event:

```
{"Event":"PROGRESS","MediaId":1,"TotalFiles":2,...}
```

can look incomplete because file two is not visible in the same object.

The design must explicitly distinguish a **command/request snapshot** from an **event update**.

Revision:

The full request contains:

```
Text
TotalFiles
Media[0]
Media[1]
...
```

while:

```
PROGRESS
```

contains only the media item whose progress changed.

This is now an explicit protocol rule rather than an implication.

Finding: numbered field names would create unnecessary branching.

A possible interpretation such as:

```
FileName1
FileName2
```

would force special-case property lookups.

Revision:

All media objects use:

```
MediaId
FileIndex
FileName
FilePath
```

This survives naturally from one file to four.

Review 1 result

The protocol now has a clean separation:

```
INPUT SNAPSHOT:
complete post

OUTPUT STREAM:
incremental state changes
```

No port architecture is needed.

Review 2

Simulated user upload: the revised Review-1 protocol is uploaded again, and I review it from a failure/recovery perspective rather than a message-shape perspective.

This review finds the more dangerous problems.

Finding: event names alone are insufficient to recover safely.

An error such as:

```
ERROR Upload failed
```

does not say whether retry is sensible, which media failed, or whether post creation might already have occurred.

Revision:

`ERROR` now has structured fields:

```
Code
Stage
Message
Retryable
MediaId       optional
FileIndex     optional
HttpStatus    optional
```

and readable codes such as:

```
MEDIA_FILE_NOT_FOUND
AUTH_FAILED
RATE_LIMITED
MEDIA_UPLOAD_FAILED
POST_CREATE_FAILED
POST_RESULT_UNKNOWN
```

Finding: automatic retries can cause duplicate posts.

A network error around the final creation request may leave Python unsure whether the remote side processed it.

Revision:

Version 1 performs **no automatic posting retry**.

A specific terminal condition exists:

```
POST_RESULT_UNKNOWN
```

which tells the operator that blindly reposting is unsafe.

Finding: terminal-event semantics were unstated.

Without an explicit rule, broken implementations could produce:

```
ERROR
PROGRESS
SUCCESS
```

or:

```
SUCCESS
ERROR
```

Revision:

Exactly one terminal application event is allowed:

```
SUCCESS
or
ERROR
```

After it, Python emits no further protocol lines.

Finding: partial uploads need an explicit policy.

Example:

```
Media 1 uploaded
Media 2 failed
```

The draft said little about what happens to `XMediaId1`.

Revision:

The whole posting operation fails.

```
do not POST
do not mark history as published
do not automatically reuse partial XMediaIds
new manual attempt = new RequestId
```

This is intentionally simple and predictable.

Finding: a Python crash cannot emit `ERROR`.

Revision:

PowerShell now owns process-level synthetic errors:

```
PYTHON_START_FAILED
PYTHON_EXITED_UNEXPECTEDLY
OPERATION_TIMEOUT
```

and protocol-level synthetic errors:

```
PROTOCOL_INVALID_STDOUT_JSON
PROTOCOL_REQUEST_ID_MISMATCH
PROTOCOL_UNEXPECTED_EVENT
PROTOCOL_SEQUENCE_ERROR
```

Thus every failure has an owner:

```
Python/application failure
    → Python ERROR event

Process/transport failure
    → PowerShell synthetic error
```

Finding: stdout and stderr separation was not strong enough.

A library stack trace or debug print could destroy the protocol framing if it appears on stdout.

Revision:

The rule becomes absolute:

```
stdout = protocol only
stderr = diagnostics only
```

and raw credentials/auth data are prohibited from both application logs and error messages.

Review 2 result

The protocol is no longer merely easy to parse. It is now safe to reason about after:

```
validation errors
authentication errors
upload failures
network failures
partial completion
protocol corruption
Python crashes
uncertain posting outcomes
```

Review 3

Simulated user upload: the second revision is uploaded again, this time reviewed as an implementation handoff: “Could another developer build this without asking us what we meant?”

The remaining issues are implementation precision rather than architectural flaws.

Finding: concurrency was still implicit.

The design discusses multiple media IDs but does not explicitly say whether Python may upload them simultaneously.

Revision:

Protocol v1 is locked to:

```
single Python process
single post request
sequential media upload
single stdout event stream
```

No parallel media execution is allowed in v1.

This also matches the existing GUI concept:

```
Uploading file 1 of 4
[=======       ] 68%

then

Uploading file 2 of 4
[===           ] 24%
```

rather than four simultaneous progress bars. filecite turn0file0

Finding: UI-thread handling needed to be stated as an implementation requirement.

Asynchronous `OutputDataReceived` callbacks should not modify GUI controls directly. Windows Forms requires cross-thread control access to be marshaled to the control's owning thread. [9]

Revision:

```
OutputDataReceived
    ↓
HandlePythonResponse
    ↓
validate/update application state
    ↓
marshal to GUI thread
    ↓
UpdateUploadProgress / UpdateGui
```

This requirement belongs in `RunPythonEngine` / `GuiController`, not in Python.

Finding: UTF-8 and line flushing needed to become mandatory rather than assumed.

JSON Lines specifies UTF-8 and line-oriented records; Python's serializers return JSON strings rather than providing a multi-record framing layer by themselves. [13]

Revision:

```
UTF-8
one object per physical line
no pretty printing
newline after every protocol object
flush after every Python event
```

Finding: configuration must not drift while coding.

The implementation should not recreate constants already finalized by the user.

Revision:

The code reads the existing:

```
Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt
```

and uses:

```
Config.PythonPath
Config.PythonScriptPath
Config.DataRootPath
Config.OperationalPath
Config.TextFilesPath
Config.MediaFilesPath
Config.ExcelFilePath
Config.KeysFilePath
Config.DebugLogReadMePath
Config.DebugLogFunctionPath
Config.DebugLogMode
Config.DebugLogOutputPath
```

without hard-coded duplicates inside functional code. This follows the final configuration and architecture supplied in the design file. filecite turn0file0

Finding: acceptance tests were needed before real X API work.

The protocol can and should first be proven using a fake Python engine.

The implementation test sequence should be:

```
Test A
Text only
READY
TEXT_COMPLETE
POSTING
SUCCESS

Test B
One fake media
READY
TEXT_COMPLETE
PROGRESS 0
PROGRESS 25
PROGRESS 50
PROGRESS 75
PROGRESS 100
MEDIA_COMPLETE
POSTING
SUCCESS

Test C
Four fake media
same sequence for MediaId 1..4
```

```
Test D
Python ERROR during MediaId 2

Test E
Python terminates without ERROR

Test F
Python writes invalid stdout text

Test G
RequestId mismatch

Test H
Empty Text + no media

Test I
Unicode/multiline Text

Test J
stderr messages while valid progress continues
```

Only after those tests pass should the fake media uploader be replaced with the actual remote upload code.

Final locked protocol

The result of all three reviews is:

```
XPoster Protocol v1
```

Transport:

```
PS1 → stdin
    exactly one compact UTF-8 JSON request line

Python → stdout
    zero or more compact UTF-8 JSON event lines
    stdout contains no non-protocol output

Python → stderr
    diagnostics only
```

Request:

```
ProtocolVersion
RequestId
Text
TotalFiles
Media[]
    MediaId
    FileIndex
    FileName
    FilePath
```

Events:

```
READY
TEXT_COMPLETE
PROGRESS
MEDIA_COMPLETE
POSTING
SUCCESS
ERROR
```

Normal state machine:

```
REQUEST
   ↓
READY
   ↓
TEXT_COMPLETE
   ↓
[ PROGRESS → MEDIA_COMPLETE ] × TotalFiles
   ↓
POSTING
   ↓
SUCCESS
```

Failure state machine:

```
any nonterminal stage
   ↓
ERROR
   ↓
stop
```

Transport/process failure:

```
PS1 detects failure
   ↓
PS1 generates local protocol/process error
   ↓
stop
```

Media identity:

```
MediaId
= local PS1↔Python identity

XMediaId
= remote upload result
```

Concurrency:

```
Sequential only in v1.
```

Recovery:

```
No automatic repost retries in v1.
```

Logging:

```
RequestId correlates the complete run.
stdout protocol events may be logged.
stderr diagnostics may be logged.
credentials/tokens/Authorization headers must never be logged.
```

Architecture:

```
PowerShell
= GUI
= application state
= workflow
= process lifecycle
= protocol parsing
= progress display
= final result handling

Python
```

```
= request validation
= credentials/API layer
= text preparation
= sequential media upload
= progress emission
= final post creation
= protocol event emission
```

This final version is concrete enough to implement without another protocol-design round, while remaining small enough that debugging a complete posting operation can be done simply by reading the ordered JSON event lines.

---

1 2 13 JSON Lines
https://jsonlines.org/

3 json — JSON encoder and decoder — Python 3.14.8 documentation
https://docs.python.org/3/library/json.html

4 11 ProcessStartInfo.RedirectStandardInput Property (System.Diagnostics) | Microsoft Learn
https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.processstartinfo.redirectstandardinput

5 Process.BeginOutputReadLine Method (System.Diagnostics) | Microsoft Learn
https://learn.microsoft.com/en-us/dotnet/api/system.diagnostics.process.beginoutputreadline?utm_source=chatgpt.com

6 subprocess — Subprocess management — Python 3.14.8 documentation
https://docs.python.org/3/library/subprocess.html

7 ConvertFrom-Json (Microsoft.PowerShell.Utility) - PowerShell | Microsoft Learn
https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/convertfrom-json

8 Process.StandardError Property (System.Diagnostics) | Microsoft Learn
https://learn.microsoft.com/dotnet/api/system.diagnostics.process.standarderror?utm_source=chatgpt.com

9 How to handle cross-thread operations with controls - Windows Forms | Microsoft Learn
https://learn.microsoft.com/en-us/dotnet/desktop/winforms/controls/how-to-make-thread-safe-calls?utm_source=chatgpt.com

10 12 ProcessStartInfo Class (System.Diagnostics) | Microsoft Learn
https://learn.microsoft.com/dotnet/api/system.diagnostics.processstartinfo?utm_source=chatgpt.com

Application Design Completion Addendum

Authoritative completion of the prototype/application requirements missing from the original PDF.

1. Locked application architecture

```
MainController
|
+-- ReadConfig
|
+-- GuiController
|   +-- DetermineStartupMode
|   +-- InitializeGui
|   +-- AttachMedia
|   +-- RemoveMedia
|   +-- UpdatePostButtonState
|   +-- UpdateGui
|   +-- UpdateUploadProgress
|
+-- StartPost
    +-- ExportPostRecord
    +-- RunPythonEngine
    +-- HandlePythonResponse
```

LOCKED: MainController owns the application workflow. GuiController owns GUI state and GUI actions. PowerShell does not implement X API details; Python owns authentication/API/media upload/post creation.

The earlier proposed function name **LoadRepostData** is not part of the final locked naming list. Its required behavior is retained and is handled by **DetermineStartupMode** + **InitializeGui** using **RepostData**.

2. Locked files and paths

```
PowerShell GUI:
Z:\InsPro\Tools\Ps1\XPoster.ps1

Python engine:
Z:\InsPro\Tools\Py\XPoster.py

Configuration:
Z:\InsPro\Tools\Data\X_Post\Operational\X_Config.txt

Credentials:
Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt

Data folders:
Z:\InsPro\Tools\Data\X_Post\TextFiles
Z:\InsPro\Tools\Data\X_Post\MediaFiles
Z:\InsPro\Tools\Data\X_Post\ExcelFile
```

Exact locked X_Config.txt content

```
PythonPath=C:\Python\Python313\python.exe
PythonScriptPath=Z:\InsPro\Tools\Py\XPoster.py
DataRootPath=Z:\InsPro\Tools\Data\X_Post
OperationalPath=Z:\InsPro\Tools\Data\X_Post\Operational
TextFilesPath=Z:\InsPro\Tools\Data\X_Post\TextFiles
MediaFilesPath=Z:\InsPro\Tools\Data\X_Post\MediaFiles
ExcelFilePath=Z:\InsPro\Tools\Data\X_Post\ExcelFile
KeysFilePath=Z:\InsPro\Tools\Data\X_Post\Operational\X_Keys.txt
DebugLogReadMePath=Z:\InsPro\Tools\Ps1\ReadMe\DebugLogReadMe.txt
DebugLogFunctionPath=Z:\InsPro\Tools\Ps1\DebugLog.ps1
DebugLogMode=Per run
DebugLogOutputPath=Z:\InsPro\Tools\Logs\XPoster\xxxxxxx.log
```

LOCKED: The literal config value is **Per run**. Do not silently rename it to PerRun in the config file. Any mapping to a logger switch is implementation behavior and must preserve the stored value.

3. Locked GUI contract

Area	Required behavior
Post text	Multiline text box. Text may be entered before or after media selection.
Attach Media	Opens file picker; stores local path(s) only; no upload before Post.
Selected media	Shows up to 4 selected filenames. State is SelectedMedia[].
Remove	Removes the chosen item from SelectedMedia[], refreshes the list, reevaluates Post-button state.
Progress	One current-file progress area. It switches from file 1 to file 2, etc.; no requirement for four simultaneous bars.
Post	Starts the local-export -> Python upload/post workflow only when enabled.
Debug / Output	Permanently visible in both prototype and final application. Receives Python/API/error/result output as appropriate.
Controls/layout	Standard Windows/WinForms controls and sensible sizing; no custom graphics requirement.

Input order is independent

```
Valid:
  Attach media -> enter text -> Post
  Enter text -> attach media -> Post
  Text only -> Post
  Media only -> Post

Invalid:
  Empty text + no selected media
```

4. Locked Post-button state rule

```
Initial state:
  Post = disabled

If PostText is not empty
OR SelectedMedia[] contains at least one item:
  Post = enabled

If PostText becomes empty
AND SelectedMedia[] becomes empty:
  Post = disabled again
```

UpdatePostButtonState must run after text changes, media attachment, and media removal. Python should also defensively reject an empty request if one somehow bypasses the GUI.

5. Locked media-selection behavior

- Attach stores local file path(s) only. Upload begins only after Post is pressed.
- SelectedMedia[] is plural from the prototype onward; no single SelectedFile design.
- Maximum selected media count is 4.
- At posting time, each selected item receives a stable local MediaId and sequential FileIndex.
- MediaId is the local PS1<->Python identity. XMediaId is a separate remote value returned after media upload.
- Uploads are sequential in protocol v1.

6. Normal startup and Repost startup

LOCKED: GuiController determines startup mode before initializing the form.

Startup mode	Behavior
Normal	Initialize an empty post form; Post starts disabled because text and media are empty.
Repost	Receive the selected saved-post data from the history/Excel mechanism; resolve the saved text and up to four media paths;

Repost is never automatic

> **Opening the application from the history/Repost action must not publish anything. It only prepares the same GUI with the prior post data. The user may edit text, add/remove media, and then manually press Post.**

The exact Excel/VBA or workbook-to-PS1 invocation mechanism is **VERIFY BEFORE CODING** unless an existing history-design file is supplied/loaded. Do not invent an XML/XSL transport merely because an earlier discussion used similar wording.

7. Local export/history contract

The history design recorded in the TXT uses a local text file per post, local media references/copies, and an Excel history view with these columns:

```
Title | Text | PostURL | Timestamp | Media 1 | Media 2 | Media 3 | Media 4
```

- Text points to the local TXT post content.
- Media 1..4 point to the saved/local media entries when present.
- PostURL is the final published X post URL when known.
- Timestamp is the publication timestamp when known.
- The history view provides the Repost action that starts the application in Repost mode for manual review.

Final save-before-Python ordering

```
User presses Post
  -> freeze the current finalized GUI values for this attempt
  -> ExportPostRecord saves the local post record FIRST
  -> only if local export succeeds: RunPythonEngine
  -> HandlePythonResponse receives realtime events
  -> SUCCESS supplies PostId/PostUrl
  -> local history fields that depend on successful publication are updated
  -> ERROR/failure leaves the already-saved local content available
```

IMPLEMENTATION DECISION: Because PostURL and publication Timestamp are not known before Python succeeds, the simplest v1 behavior is to create the local record with those publication-result fields blank, then fill them on SUCCESS. On failure they remain blank. This adds no new history column and preserves the locked rule that the finalized local content is saved before any network operation.

VERIFY BEFORE CODING: whether media are copied into MediaFilesPath or merely referenced from their original location. The source design allows local copies/references but does not fully lock this policy.

8. Complete Post runtime flow

```
MainController
  -> ReadConfig
  -> GuiController
       -> DetermineStartupMode
       -> InitializeGui
  -> Idle / user interaction
       -> AttachMedia / RemoveMedia / text changes
       -> UpdatePostButtonState
       -> UpdateGui

User presses Post
  -> block duplicate Post and prevent unsafe input changes
  -> capture PostText + SelectedMedia[]
  -> ExportPostRecord FIRST
       -> failure here: stop; do not start Python
  -> build one complete JSON request
  -> RunPythonEngine
       -> one Python process
       -> request through stdin
       -> JSONL protocol events through stdout
       -> diagnostics through stderr
  -> HandlePythonResponse
       -> READY
       -> TEXT_COMPLETE
       -> for each media sequentially:
            PROGRESS ...
            MEDIA_COMPLETE
       -> POSTING
       -> SUCCESS or ERROR
  -> GuiController.UpdateUploadProgress / UpdateGui
  -> restore correct GUI state when process finishes
```

Only a failure stops the posting sequence after the local export has succeeded. There are no sockets and no per-file ports.

9. Protocol integration rules retained from the original PDF

Rule	Status
One complete request sent once through stdin	LOCKED
Python stdout is protocol JSON Lines only	LOCKED
Python stderr is diagnostics/debug only	LOCKED
Request contains Text, TotalFiles and all Media entries	LOCKED
PROGRESS describes only the currently changing media item	LOCKED
Stable RequestId echoed in events	LOCKED
Stable local MediaId distinct from XMediaId	LOCKED
Seven event names: READY, PROGRESS, TEXT_COMPLETE, MEDIA_COMPLETE, POSTING, SUCCESS, ERROR	LOCKED
Media upload is sequential	LOCKED
No automatic repost retry after uncertain final-post outcome	IMPLEMENTATION SAFETY DECISION

Progress accuracy rule: production progress must be truthful; actual X upload progress semantics remain a VERIFY BEFORE CODING item and are listed in Section 11.

10. Updated implementation work plan

Phase	Action
Phase 0 - Source lock	Use this revised PDF + final TXT decisions as the implementation baseline. Do not redesign locked names/paths/flow.
Phase 1 - Verify dependencies	Read X_Config.txt; verify DebugLog.ps1 contract/readme; verify credential-file format; verify Python 3.13 runtime; verify X API aut
Phase 2 - PS1 GUI shell	Implement WinForms controls: multiline text, Attach, selected-media list, Remove, progress area, Post button, permanent Debug/
Phase 3 - Startup modes	Implement DetermineStartupMode + InitializeGui for Normal and Repost. Repost populates but never posts automatically.
Phase 4 - Local export/history	Implement ExportPostRecord so local save occurs before Python. Implement TXT/media/history handling and the existing history
Phase 5 - Protocol harness	Implement Python request reader/event emitter using fake media/post actions. Implement PS1 process launch, async stdout/stder
Phase 6 - Realtime progress state	Wire CurrentFileIndex, TotalFiles, CurrentFileProgress, CurrentFileName and per-media completion mapping. Prove sequential sw
Phase 7 - Real X engine	After protocol harness passes, add credential loading/auth/token handling, media upload, actual truthful progress/milestones, and
Phase 8 - Success/history update	On SUCCESS store PublishedPostId/PublishedPostUrl and update the publication-dependent history fields. Do not destroy the pr
Phase 9 - Excel/Repost integration	Wire the existing history/Repost mechanism to launch PS1 with selected record data, using the already approved manual-review l
Phase 10 - Fault testing	Exercise malformed JSON, Python crash, stderr traffic, media failure, auth/network failure, protocol sequence error, request-ID mi
Phase 11 - Final acceptance	Run syntax, logic, logging, GUI-state, protocol, Repost, local-history and real-post tests. Review each category three times before

11. Minimum verification items before production coding

Item	Why it remains open	Required evidence
X_Keys.txt exact schema	TXT records the path but not a final field schema.	Read/approve actual credentials file format before loader code.
X API auth + media endpoint compatibility	Patch only design cannot prove which auth mode is accepted by the selected X endpoints.	Verify against current X API documentation/account capabilities.
Realtime progress semantics	A true percentage depends on the HTTP client/API upload behavior.	Prototype actual upload and determine whether byte/chunk prog
DebugLog.ps1 exact call contract	Config points to the existing logger, but implementation must match the existing function names/switches.	Read DebugLog.ps1 before coding lo
Excel/Repost invocation	Behavior is approved; the transport/invocation details are not fully defined in the existing history design/workbook specification before w	Load the existing history design/workbook specification before w
Media copy vs reference policy	The design uses local media history but does not fully lock whether files are copied.	Choose/confirm the simplest policy before ExportPostRecord imp

12. Acceptance test matrix

ID	Scenario	Expected
GUI-01	Initial empty form	Post disabled.
GUI-02	Enter text only	Post enabled.
GUI-03	Clear text, attach media	Post remains enabled.
GUI-04	Remove last media while text empty	Post disabled again.
GUI-05	Attach 4 media	All displayed; fifth is rejected/blocked by max=4.
GUI-06	Remove one media	SelectedMedia[] and visible list refresh; Post state reevaluated.
REPOST-01	Launch with saved record	GUI prepopulates text/media and waits for manual Post.
EXPORT-01	Press Post with valid local content	ExportPostRecord completes before Python starts.
EXPORT-02	Local export fails	Python is not started; error is logged/displayed.
PROTO-01	Text-only fake request	READY -> TEXT_COMPLETE -> POSTING -> SUCCESS.
PROTO-02	One fake media	Sequential truthful PROGRESS -> MEDIA_COMPLETE -> POSTING -> SUCCESS.
PROTO-03	Four fake media	FileIndex/MediaId switch 1->2->3->4; one current progress display.
PROTO-04	stderr during valid run	Debug/output/log receives diagnostics; stdout parser remains intact.
PROTO-05	Invalid stdout JSON	Operation terminates as protocol failure.
PROTO-06	RequestId mismatch	Wrong/stale event cannot update current GUI operation.
FAIL-01	Media 2 upload failure	No POSTING/SUCCESS; local record remains available.
FAIL-02	Python crash	PS1 records process-level failure and restores GUI safely.
FAIL-03	Uncertain final post result	No automatic resend that could duplicate a post.
DATA-01	Unicode + multiline text	JSON remains one physical line; text round-trips correctly.
LOG-01	Normal + error runs	Meaningful stages/errors logged; credentials/tokens never logged.
HIST-01	SUCCESS	PostURL/publication Timestamp fields are updated in the existing record.
HIST-02	ERROR after local export	Pre-saved text/media record remains; result-dependent fields stay unfilled.

13. Three-pass review of this corrected specification

Review Pass 1 - Coverage against final TXT

- Checked GUI requirements: multiline text, Attach, up to four media, Remove, one current progress display, permanent Debug/Output, standard controls - now explicitly covered.
- Checked Post-button state transitions - now explicitly covered.
- Checked Normal/Repost startup and manual Repost - now explicitly covered without reintroducing the superseded LoadRepostData function name into the final function list.
- Checked final save-before-Python rule - now explicit and placed before RunPythonEngine.
- Checked exact X_Config.txt values and paths - preserved verbatim, including DebugLogMode=Per run.

Pass 1 result: no previously identified application-level requirement is left implicit.

Review Pass 2 - Contradictions and superseded decisions

- Earlier export-after-success wording is explicitly superseded by the later approved export-before-Python rule.
- Earlier LoadRepostData proposal is treated as historical; the final locked function list remains unchanged while its behavior is assigned to DetermineStartupMode/InitializeGui.
- The document does not claim XML/XSL as the active design when the TXT records CSV/TXT/Excel history behavior.
- The local history scheme does not fabricate PostURL/Timestamp before success; publication-result fields are left blank and filled on SUCCESS as an implementation decision.
- Protocol safety additions such as detailed error taxonomy are clearly distinguished from user-locked design where appropriate.

Pass 2 result: the revised document has a clear authority order, so exploratory text cannot silently override final decisions.

Review Pass 3 - Implementation handoff

- A developer can now implement the GUI and state rules without consulting earlier conversation fragments.
- The exact Post runtime order is explicit, including local export failure behavior.
- Repost behavior is executable: populate, review/edit, manual Post.
- The protocol boundary remains testable independently with a fake Python engine before real X API work.
- External unknowns are isolated in a short VERIFY BEFORE CODING list instead of being silently guessed.
- Acceptance tests cover syntax/protocol, GUI logic, local export/history, Repost, error logging, process failure and real-post integration.

Pass 3 result: this revised PDF is suitable as the single implementation work-plan/specification, provided the listed external/API items are verified before the corresponding production code is written.

14. Final implementation gate

```
DO NOT start real X posting code until:
  [ ] X_Config.txt is readable and validated
  [ ] DebugLog.ps1 contract has been read and matched
  [ ] X_Keys.txt schema is known
  [ ] PS1 GUI/state behavior passes local tests
  [ ] Normal and Repost startup both pass
  [ ] ExportPostRecord demonstrably runs before Python
  [ ] Fake JSONL protocol harness passes all protocol tests
  [ ] stdout/stderr separation is proven
  [ ] GUI remains responsive during streamed output
  [ ] actual X auth/media/post endpoints are verified
  [ ] production progress is truthful, not fabricated

Only then replace fake remote operations with the real X API engine.
```